

---

# Evaluating Human Feature Analysis for Style Transfer

---

Anonymous Author

## 1 Human Feature Analysis

### 1.1 Human Patch Extraction

We use YOLO26m [1] to detect and extract 4000 human patches from the training videos. The budget is divided equally between domains (2,000 each); within the movie domain it is allocated proportional to video duration so that longer clips contribute more patches. To prioritise diverse patches, we reject near-static frames and enforce a minimum spacing between YOLO calls (Table 1).

Table 1: Extraction hyperparameters (empirically chosen).

| Parameter                           | Value | Role                                                             |
|-------------------------------------|-------|------------------------------------------------------------------|
| yolo_interval, $\delta_f$           | 10    | minimum frames between YOLO calls                                |
| scene_change_threshold, $\tau_{sc}$ | 8     | minimum inter-frame MAD; rejects near-static shots               |
| blur_threshold, $\tau$              | 40    | Laplacian blur floor; normalised as $\min(\text{blur}/3\tau, 1)$ |

Table 2: Composite patch quality score components. Maximum achievable score is 0.75; sharpness is normalised against  $3\tau$  (Table 1).

| Component           | Formula                        | Weight |
|---------------------|--------------------------------|--------|
| YOLO confidence     | $c \in [0, 1]$                 | 0.30   |
| Relative size       | soft peak at 15% of frame area | 0.25   |
| Laplacian sharpness | $\min(\text{blur}/3\tau, 1)$   | 0.20   |

Candidate patches are scored as in Table 2, and Figure 1 shows the results. Greedy diversity sampling enforces a  $\delta_f$ -frame temporal gap per video source.

### 1.2 Classification

We classify patches using a 3-layer Graph Convolutional Network (GCN) [2; 3] on COCO [4] skeleton graphs with node features  $(x, y, \text{confidence})$ , and confidence-weighted global pooling [5]. We use YOLO26m-pose [1] to detect pose keypoints for the skeletons. Table 3 shows the results.

GCN training labels are manually annotated using a custom tool (Figure 2) seeded with rule-based keypoint heuristics (Table 5), which are less flexible than the learned GCN classifier. Horizontal flip augmentation (swapping symmetric keypoint pairs) doubles the GCN training set without requiring additional annotations.

Figure 4 shows that manual annotations generally improve accuracy over rule-based labels, notably except for Others (Table 4).

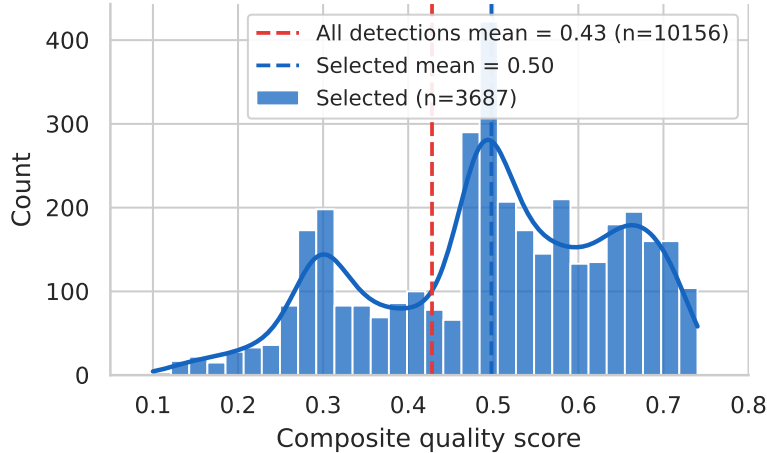


Figure 1: Distribution of composite quality scores for the 3,687 selected patches. Scores combine YOLO detection confidence, relative patch size, and Laplacian sharpness (weights in Table 2). The selected mean (0.50) lies above the mean of all 10,156 raw detections (0.43), suggesting that the scoring criterion prioritises sharper, well-sized, high-confidence crops.

Table 3: GCN result on the entire dataset: patch distribution across classes. No pose means that the pose model didn’t detect anything, which is kept separate from Others to improve GCN performance on Others.

| Class               | Count | %     |
|---------------------|-------|-------|
| Head-shoulder front | 1172  | 31.8  |
| Head-shoulder back  | 1024  | 27.8  |
| Full-body back      | 700   | 19.0  |
| Full-body front     | 435   | 11.8  |
| Others              | 202   | 5.5   |
| No pose             | 154   | 4.2   |
| Total               | 3687  | 100.0 |

### 1.3 Training Data Selection

Many extracted patches are visually similar, so we select diverse training patches as follows:

1. DINOv2 ViT-B/14 [6; 7; 8] embedding of all non-others patches to cluster by visual similarity.
2. Apply K-Means clustering in each class  $\times$  domain group. The 2,000-patch budget is split equally, with  $k \propto$  group size within each domain.
3. Select the centroid-nearest patch by cosine similarity (i.e. the most representative sample per cluster).

Figure 5 shows the DINOv2 embeddings are not visually separable by class (left), but they are somewhat by domain (right). This suggests GCN skeletal features are more important than visual similarity.

## 2 Real-world Application

### 2.1 Image Model Deployment

For our baseline, we use a pre-trained horse2zebra Contrastive Unpaired Translation (CUT) model [9], and fine-tune it on full frames from the training videos (1000 frames per domain; 2000 in total).

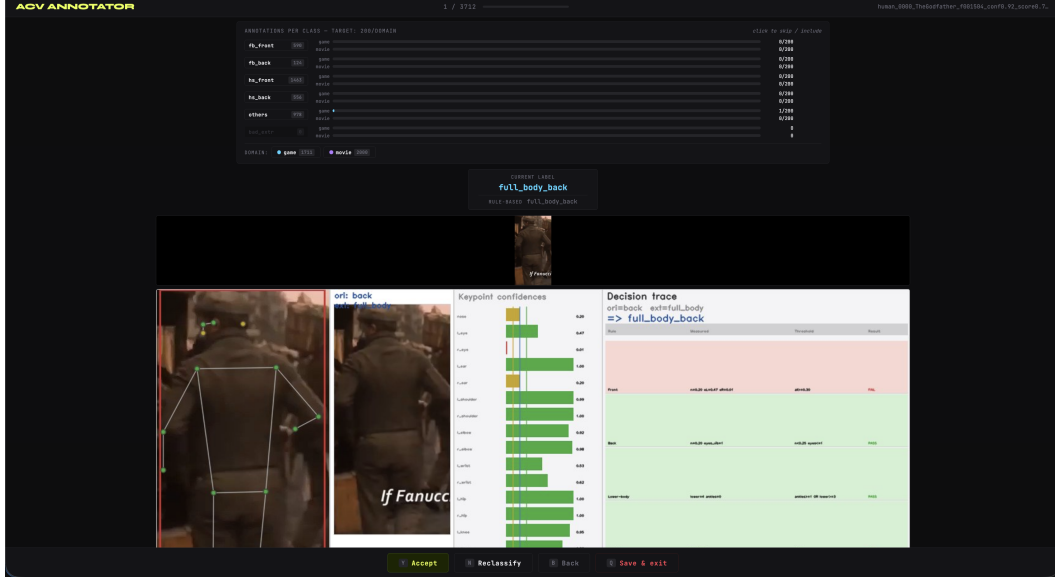


Figure 2: Custom tool for manual annotation of extracted patches. In this screenshot, the user is prompted to accept a rule-based heuristic classification or reclassify.

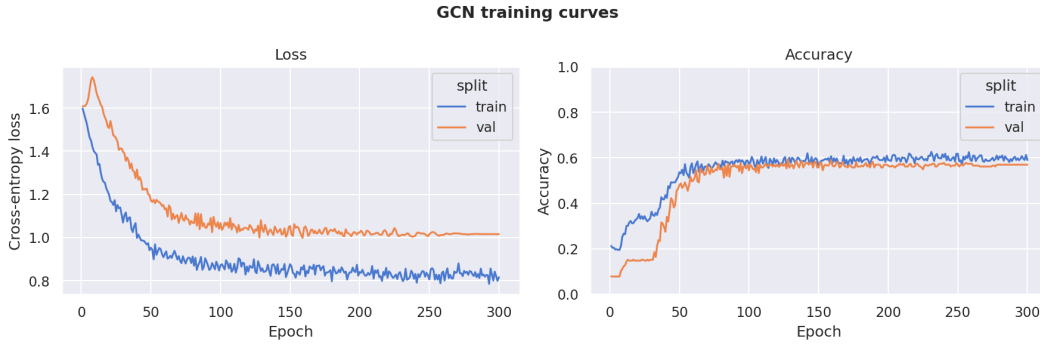


Figure 3: Final GCN training and validation curves using manual annotations.

The experimental setup is shown in Table 6. CUT uses multilayer, patchwise contrastive loss which is more effective than cycle consistency loss (e.g. CycleGAN [10; 11]), and CUT has only one generator and one discriminator which makes it  $\sim 50\%$  more efficient than CycleGAN [9].

Table 7 shows FID, KID [12] and LPIPS [13] for both translation directions. Movie  $\rightarrow$  game translation performs worse for FID (271.3 vs 239.4) and KID (0.105 vs 0.085), which suggests that the opposite direction may be easier to learn, but the better LPIPS score (0.286 vs 0.391) suggests that the model more aggressively restyles in the game  $\rightarrow$  movie direction.

Figures 6 and 7 show that game  $\rightarrow$  movie transfer results in heavy blurring and a green tinge, which may be an artifact from the horse2zebra initialisation. However, movie  $\rightarrow$  game style transfer sometimes works well (Figure 9) but occasionally fails or incorrectly distorts the image (Figure 8).

The pre-trained horse2zebra weights are tuned for animal textures not humans, which is a large domain gap. Results are plausible for 5 epochs of fine-tuning (no zebra stripes!), but longer fine-tuning on a more appropriate pre-trained base may improve results.

Table 4: GCN per-class validation accuracy on the manually annotated validation set. Poor performance on `Others` is expected, because the class contains every detected pose that does not fit into the other 4 classes and is therefore very heterogeneous, so the model struggles to learn patterns.

| Class               | Correct | Total | Acc. (%) |
|---------------------|---------|-------|----------|
| Full-body front     | 26      | 32    | 81.2     |
| Full-body back      | 8       | 10    | 80.0     |
| Head-shoulder front | 19      | 36    | 52.8     |
| Head-shoulder back  | 9       | 10    | 90.0     |
| Others              | 0       | 18    | 0.0      |

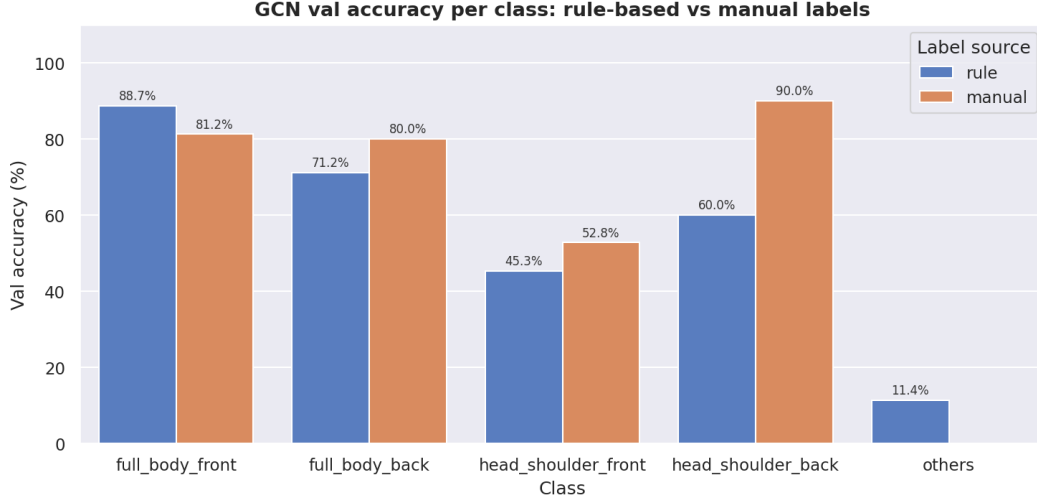


Figure 4: Per-class GCN validation accuracy when trained on rule-based vs. manually annotated labels.

## 2.2 Local (Temporal) Enhancement

We improve the baseline in Section 2.1 using human feature analysis from Section 1 and temporal enhancements. We fine-tune the pre-trained `horse2zebra` CUT model on the patches from Section 1.3, using the same setup in Table 6.

During test inference, we:

1. Extract human patches as in Section 1.1, except with more lenient  $\tau = 10$  and both  $\tau_{sc} = 0$  and  $\delta_f = 1$  to cover every test frame (Table 1).
2. Classify patches using a spatio-temporal extension of the Section 1.2 GCN (ST-GCN, [14]) over a  $T = 5$  frame window per tracked person, and discard `Others` to reduce noise (Table 8).
3. Apply the fine-tuned CUT model to the remaining patches.
4. Resize and paste the translated patches back onto their respective frames. Use a  $21 \times 21$  Gaussian kernel ( $\sigma = 5$ ) for feather blending to mitigate obvious seams.
5. Apply exponential moving average (EMA) temporal blending ( $\alpha = 0.3$ ) with `cv2.addWeighted` [15] between consecutive frames to reduce flicker.

Table 7 shows that this method is an overall improvement, producing lower FID, KID and LPIPS scores for both directions. This implies that patch-level training significantly improves performance over full-frame training. Without feathering and EMA blending, the FID and KID were much higher because of obvious seams after pasting. These artifacts are still present in Figure 10, so improved blending (e.g. pyramid blending) may further improve results.



Table 5: Rule-based heuristics used to seed manual annotation, applied sequentially. Thresholds:  $\tau_f=0.30$  (face),  $\tau_n=0.25$  (nose-back),  $\tau_b=0.10$  (eye-back),  $\tau_{bd}=0.20$  (body).

| Step        | Condition                                                                                     | Outcome       |
|-------------|-----------------------------------------------------------------------------------------------|---------------|
| Orientation | $\text{nose}, \text{l\_eye}, \text{r\_eye} \geq \tau_f$                                       | front         |
|             | $\text{nose} < \tau_n, \text{eyes visible (conf} \geq \tau_b) \leq 1$                         | back          |
|             | otherwise                                                                                     | <i>others</i> |
| Extent      | $\text{ankles} \geq 1$ or $\geq 3$ lower kps ( $\text{conf} \geq \tau_{bd}$ ), $h/w \geq 1.5$ | full-body     |
|             | $\geq 1$ shoulder visible ( $\text{conf} \geq \tau_{bd}$ )                                    | head-shoulder |
|             | otherwise                                                                                     | <i>others</i> |

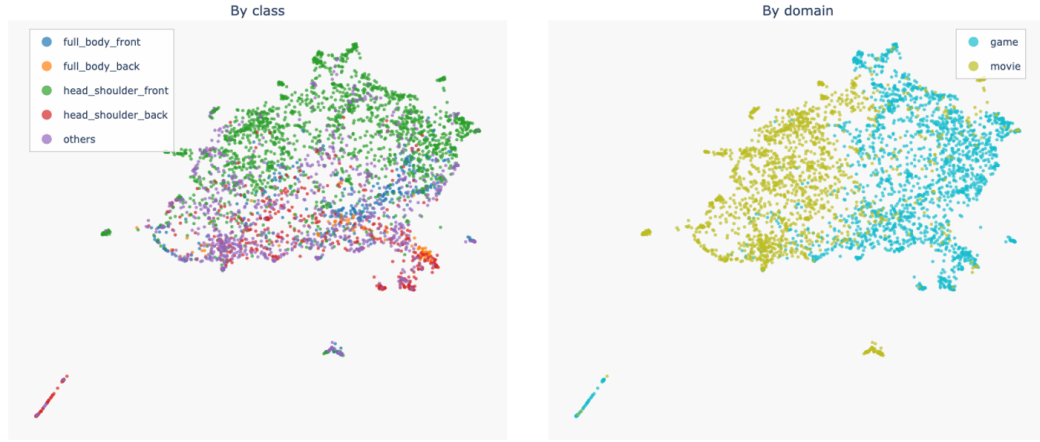


Figure 5: UMAP projection of DINOv2 embeddings for all extracted patches, coloured by pose class as in Section 1.2 (left) and domain (right). The embeddings are not visually separable by class, but they are somewhat by domain. This suggests GCN skeletal features are more important than visual similarity.

However, Figure 11 shows that the enhanced method mostly failed to learn any movie  $\rightarrow$  game transfer, and sometimes the baseline looks better. This may be because there are 3 different movies (compared to 1 game), and more epochs are needed to learn the distribution.



Figure 6: Game  $\rightarrow$  movie failures using the baseline method.



Figure 7: Game  $\rightarrow$  movie successes using the baseline method.



Figure 8: Movie  $\rightarrow$  game failures using the baseline method. The frames are distorted and not game-like.





Figure 9: Movie → game successes using the baseline method. The frames gain a sepia tint as seen in the Mafia videogame training videos.

Table 6: Finetuning setup shared by both Section 2.1 (full-frame CUT) and Section 2.2 (patch-level CUT). Training data differs: 2.1 uses full frames, whereas 2.2 uses human patches selected in Section 1.3.

| Hyperparameter      | Value                                       |
|---------------------|---------------------------------------------|
| Pretrained weights  | horse2zebra CUT [9]                         |
| Constant-LR epochs  | 4                                           |
| LR-decay epochs     | 1                                           |
| Batch size          | 4                                           |
| Load size           | $286 \times 286$                            |
| Crop size           | $256 \times 256$                            |
| Learning rate       | $2 \times 10^{-4}$ (Adam, $\beta_1 = 0.5$ ) |
| GPU                 | NVIDIA GeForce RTX 2080 Ti                  |
| Training time (2.1) | 13m 4s                                      |
| Training time (2.2) | 30m 10s                                     |
| Frames per domain   | 1000                                        |

Table 7: Metric comparison between 2.1 (full-frame CUT) and 2.2 (patch CUT + ST-GCN) for both translation directions. FID/KID measure distance to the target-domain distribution; LPIPS measures perceptual change versus the input. For all metrics, lower is considered better. Bold indicates the better score per direction.

| Model                  | Direction    | FID ↓        | KID ↓        | LPIPS ↓      |
|------------------------|--------------|--------------|--------------|--------------|
| 2.1 Full-frame CUT     | Game → Movie | 239.4        | 0.085        | 0.391        |
|                        | Movie → Game | 271.3        | 0.105        | 0.286        |
| 2.2 Patch CUT + ST-GCN | Game → Movie | <b>186.3</b> | <b>0.063</b> | <b>0.317</b> |
|                        | Movie → Game | <b>201.5</b> | <b>0.083</b> | <b>0.271</b> |

## Disclosure

The codebase was written with the assistance of Claude Code, but the report was not.



Figure 10: Game  $\rightarrow$  movie: comparison between original (left), 2.1 baseline (center) and 2.2 enhanced pipeline (right). In these frames, the enhanced method effectively targets the people in the frames, whereas the baseline method creates a blur with green tinge.





Figure 11: Movie  $\rightarrow$  game: comparison between original (left), 2.1 baseline (center) and 2.2 enhanced pipeline (right). In these frames, the baseline generally increases contrast and adds a sepia tinge, whereas the enhanced method looks very similar to the original, suggesting it has failed to learn movie  $\rightarrow$  game style transfer. Although the baseline sometimes distorts the images, it arguably performs better than the enhanced method here.



Table 8: Inference-time parameters specific to Section 2.2 (not covered by Table 6).

| Parameter                 | Value                      | Role                                    |
|---------------------------|----------------------------|-----------------------------------------|
| Blur threshold $\tau$     | 10                         | Lenient floor for test patch acceptance |
| Patch size                | $256 \times 256$           | Crop resize before CUT inference        |
| Feather kernel            | $21 \times 21, \sigma = 5$ | Gaussian mask blended at bbox border    |
| EMA blend weight $\alpha$ | 0.3                        | Temporal blend strength (0 = off)       |
| ST-GCN window $T$         | 5 frames                   | Spatio-temporal classification window   |
| Excluded classes          | others                     | GCN classes skipped during compositing  |

## References

- [1] Glenn Jocher and Jing Qiu. Ultralytics yolo26, 2026. URL <https://github.com/ultralytics/ultralytics>.
- [2] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks, 2017. URL <https://arxiv.org/abs/1609.02907>.
- [3] Sijie Yan, Yuanjun Xiong, and Dahua Lin. Spatial temporal graph convolutional networks for skeleton-based action recognition. *Proceedings of the AAAI Conference on Artificial Intelligence*, 32(1), Apr. 2018. doi: 10.1609/aaai.v32i1.12328. URL <https://ojs.aaai.org/index.php/AAAI/article/view/12328>.
- [4] Tsung-Yi Lin, Michael Maire, Serge Belongie, Lubomir Bourdev, Ross Girshick, James Hays, Pietro Perona, Deva Ramanan, C. Lawrence Zitnick, and Piotr Dollár. Microsoft coco: Common objects in context, 2015.
- [5] Yuanming Hu, Baoyuan Wang, and Stephen Lin. FC<sup>4</sup>: Fully convolutional color constancy with confidence-weighted pooling. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 330–339, 2017. doi: 10.1109/CVPR.2017.43.
- [6] Maxime Oquab, Timothée Darcet, Theo Moutakanni, Huy V. Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, Russell Howes, Po-Yao Huang, Hu Xu, Vasu Sharma, Shang-Wen Li, Wojciech Galuba, Mike Rabbat, Mido Assran, Nicolas Ballas, Gabriel Synnaeve, Ishan Misra, Herve Jegou, Julien Mairal, Patrick Labatut, Armand Joulin, and Piotr Bojanowski. Dinov2: Learning robust visual features without supervision, 2023.
- [7] Timothée Darcet, Maxime Oquab, Julien Mairal, and Piotr Bojanowski. Vision transformers need registers, 2023.
- [8] Cijo Jose, Théo Moutakanni, Dahyun Kang, Federico Baldassarre, Timothée Darcet, Hu Xu, Daniel Li, Marc Szafraniec, Michaël Ramamonjisoa, Maxime Oquab, Oriane Siméoni, Huy V. Vo, Patrick Labatut, and Piotr Bojanowski. Dinov2 meets text: A unified framework for image- and pixel-level vision-language alignment, 2024.
- [9] Taesung Park, Alexei A. Efros, Richard Zhang, and Jun-Yan Zhu. Contrastive learning for unpaired image-to-image translation. In *European Conference on Computer Vision*, 2020.
- [10] Jun-Yan Zhu, Taesung Park, Phillip Isola, and Alexei A Efros. Unpaired image-to-image translation using cycle-consistent adversarial networks. In *Computer Vision (ICCV), 2017 IEEE International Conference on*, 2017.
- [11] Phillip Isola, Jun-Yan Zhu, Tinghui Zhou, and Alexei A Efros. Image-to-image translation with conditional adversarial networks. In *Computer Vision and Pattern Recognition (CVPR), 2017 IEEE Conference on*, 2017.
- [12] Gaurav Parmar, Richard Zhang, and Jun-Yan Zhu. On aliased resizing and surprising subtleties in gan evaluation. In *CVPR*, 2022.
- [13] Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *CVPR*, 2018.

- [14] Sijie Yan, Yuanjun Xiong, and Dahua Lin. Spatial temporal graph convolutional networks for skeleton-based action recognition. In *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence and Thirtieth Innovative Applications of Artificial Intelligence Conference and Eighth AAAI Symposium on Educational Advances in Artificial Intelligence, AAAI'18/IAAI'18/EAAI'18*. AAAI Press, 2018. ISBN 978-1-57735-800-8.
- [15] G. Bradski. The OpenCV Library. *Dr. Dobb's Journal of Software Tools*, 2000.